

LAB # 05

JOINING TABLES USING FOREIGN KEY

Objective:

To create two related tables in SQL using a Foreign Key and to display their combined data using a JOIN query.

Introduction:

A foreign key is a column in one table that refers to the primary key of another table, creating a defined relationship between the two. It allows logically connected data, such as a student and the fee record of that student, to be stored in separate, well-structured tables instead of being repeated in one large table.

The JOIN clause is used to query multiple related tables together by matching rows on a shared key. In this lab, a students table and a fees table were created and linked through a foreign key on student_id. They were then queried together using JOIN to show each student's academic and fee information in a single result.

- **PRIMARY KEY** : uniquely identifies each row of a table.
- **FOREIGN KEY** : links a column of one table to the primary key of another table.
- **JOIN** : combines rows from two or more tables based on a matching column.

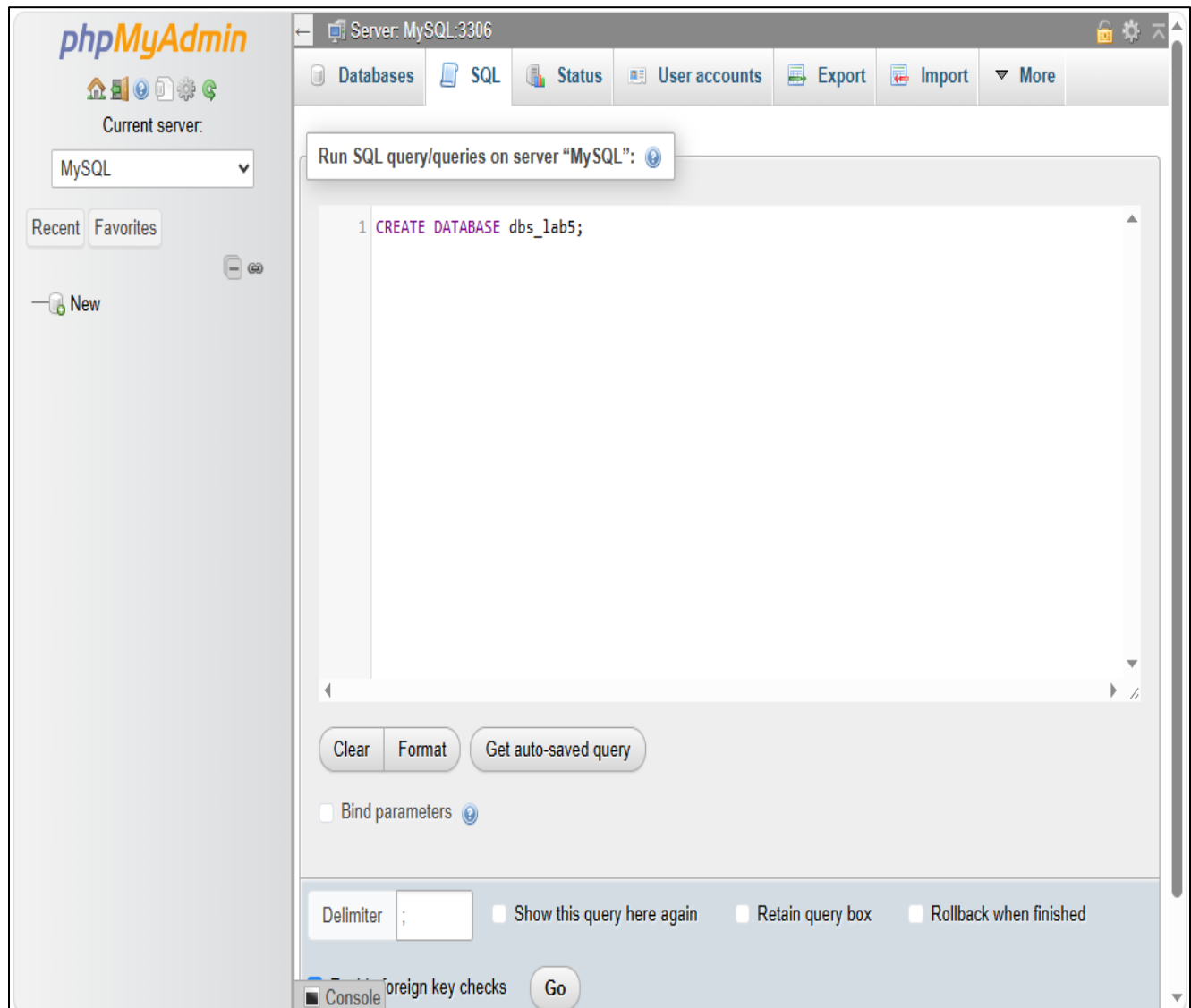
Procedure:

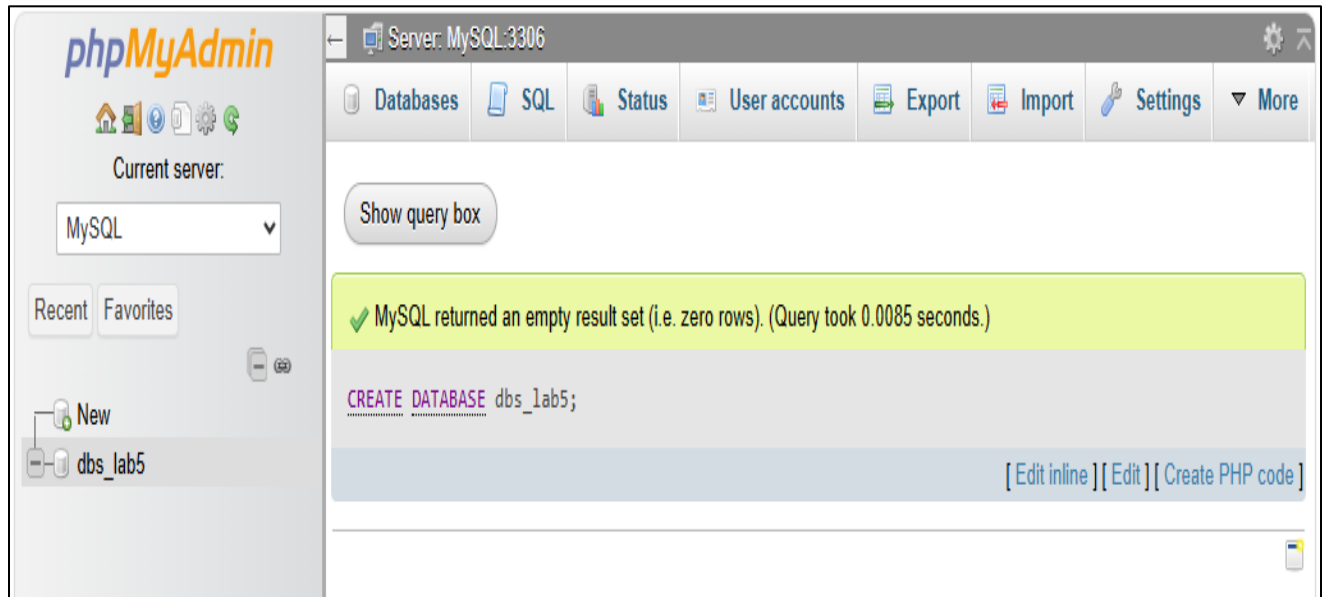
1. Start WampServer and open phpMyAdmin in the browser (localhost/phpmyadmin).
2. Create the database dbs_lab5 and select it.
3. Create the students table with student_id as the primary key and insert four records.
4. Create the fees table with a foreign key on student_id and insert four records.
5. Run the JOIN query, then add the WHERE conditions one by one in the SQL tab.
6. Take a screenshot of each query and of its output.

LAB TASK**Step 1: Create Database**

A database named "dbs_lab5" was created and then selected for use.

Query: CREATE DATABASE dbs_lab5;



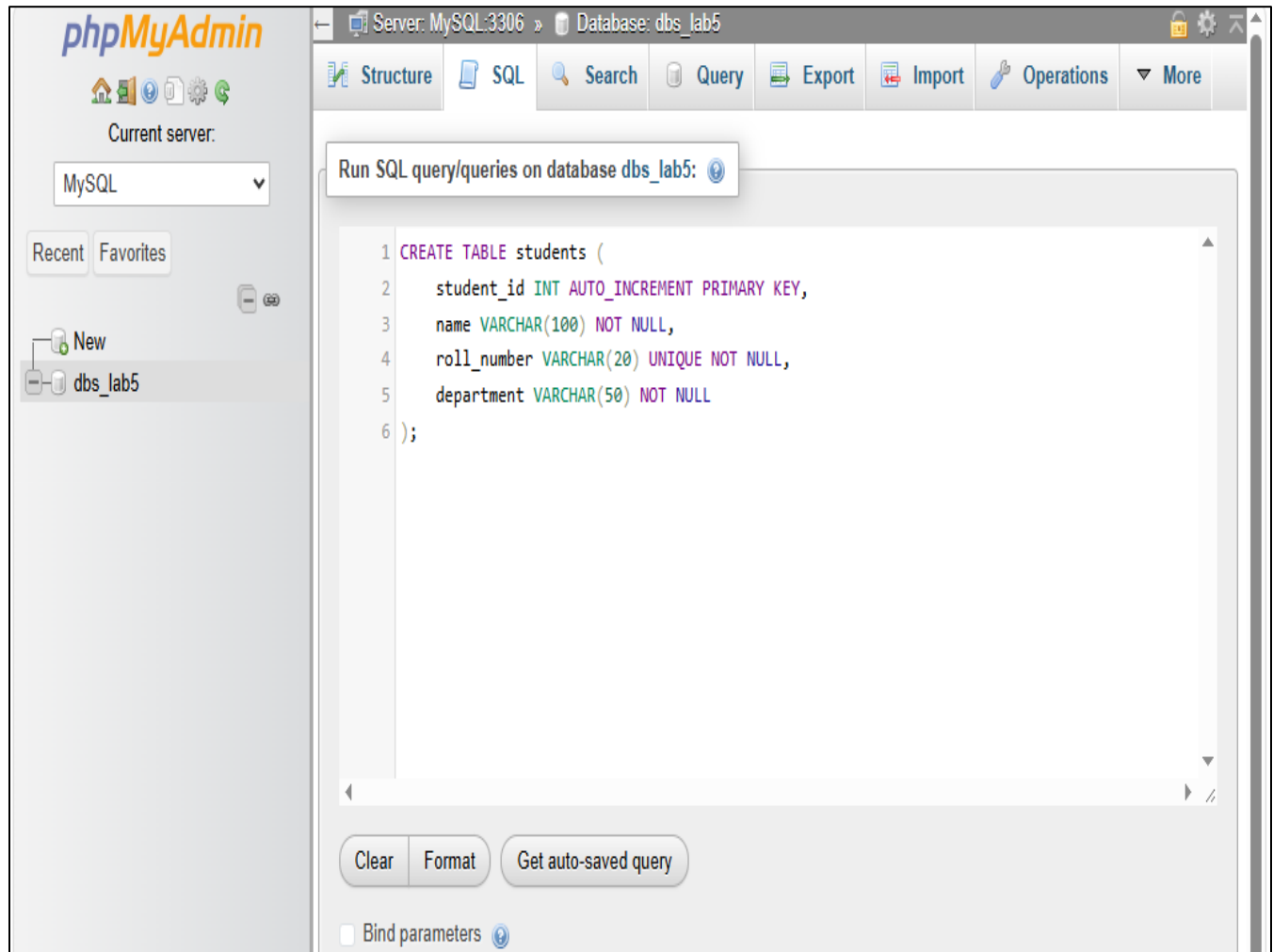


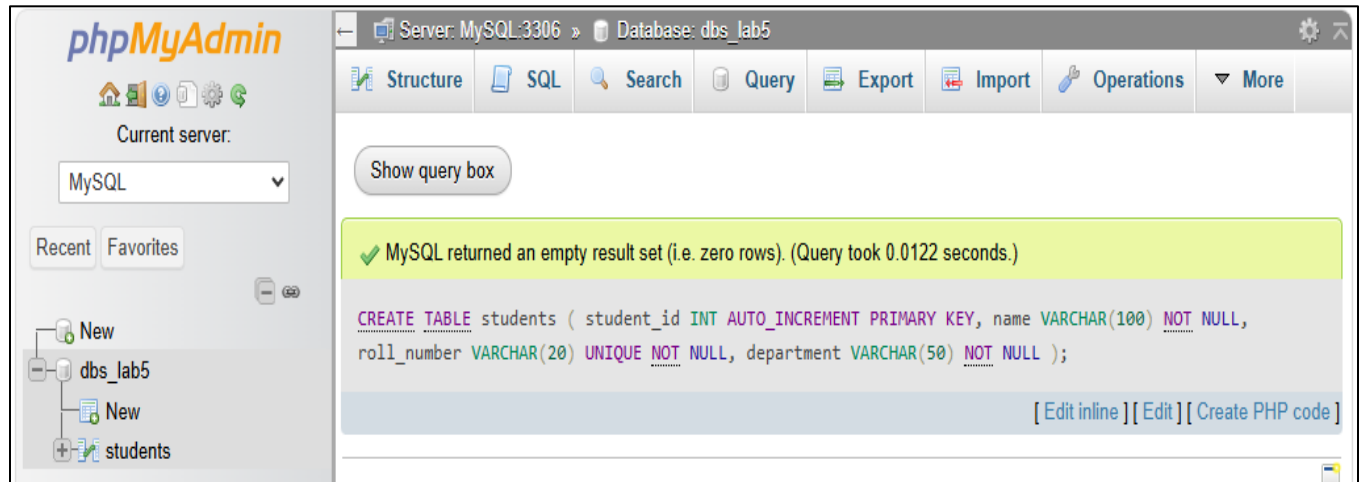
Output: Database created successfully

Step 2: Create the students Table

The students table was created with student_id as the primary key.

Query: *CREATE TABLE students (student_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, roll_number VARCHAR(20) UNIQUE NOT NULL, department VARCHAR(50) NOT NULL);*





Output: students table created successfully

Step 3: Insert Records into students

Four student records were inserted into the students table.

Query: INSERT INTO students (name, roll_number, department) VALUES ('Hamza', 'CY101', 'Cyber Security'), ('Sara', 'CS102', 'Computer Science'), ('Bilal', 'EE103', 'Electrical Engineering'), ('Fatima', 'CY104', 'Cyber Security');

The screenshot shows the phpMyAdmin web interface. The top navigation bar indicates the current context: Server: MySQL:3306, Database: dbs_lab5, and Table: students. The main toolbar includes options like Browse, Structure, SQL, Search, Insert, Export, Import, and More. The left sidebar shows the database structure with 'dbs_lab5' selected and the 'students' table highlighted. The central area is titled 'Run SQL query/queries on table dbs_lab5.students:' and contains a text editor with the following SQL query:

```
1 INSERT INTO students (name, roll_number, department)
  VALUES
2 ('Hamza', 'CY101', 'Cyber Security'),
3 ('Sara', 'CS102', 'Computer Science'),
4 ('Bilal', 'EE103', 'Electrical Engineering'),
5 ('Fatima', 'CY104', 'Cyber Security');
```

To the right of the query editor is a preview of the table structure:

student_id
name
roll_number
department

Below the query editor are buttons for 'SELECT *', 'SELECT', 'INSERT', 'UPDATE', 'DELETE', 'Clear', 'Format', and '<<'. There is also a 'Get auto-saved query' button and a checkbox for 'Bind parameters'.

The screenshot shows the phpMyAdmin interface with the 'students' table selected in the 'db5_lab5' database. A yellow message box indicates that 4 rows were successfully inserted. Below the message, the SQL query is displayed: `INSERT INTO students (name, roll_number, department) VALUES ('Hamza', 'CY101', 'Cyber Security'), ('Sara', 'CS102', 'Computer Science'), ('Bilal', 'EE103', 'Electrical Engineering'), ('Fatima', 'CY104', 'Cyber Security');`. The interface includes a sidebar with a tree view showing the database structure and a top navigation bar with options like Browse, Structure, SQL, Search, Insert, Export, Import, and Privileges.

Server: MySQL:3306 » Database: db5_lab5 » Table: students

Current server: MySQL

Recent Favorites

New

db5_lab5

New

students

Show query box

✓ 4 rows inserted.
Inserted row id: 4 (Query took 0.0080 seconds.)

```
INSERT INTO students (name, roll_number, department) VALUES ('Hamza', 'CY101', 'Cyber Security'), ('Sara', 'CS102', 'Computer Science'), ('Bilal', 'EE103', 'Electrical Engineering'), ('Fatima', 'CY104', 'Cyber Security');
```

[Edit inline] [Edit] [Create PHP code]

The screenshot shows the phpMyAdmin interface displaying the results of a `SELECT * FROM students;` query. A yellow message box indicates that 4 rows are shown. Below the message, the query is displayed. The results are shown in a table with columns: student_id, name, roll_number, and department. The table contains 4 rows of data. The interface includes a sidebar with a tree view showing the database structure and a top navigation bar with options like Browse, Structure, SQL, Search, Insert, Export, Import, and Privileges.

Server: MySQL:3306 » Database: db5_lab5 » Table: students

Current server: MySQL

Recent Favorites

New

db5_lab5

New

students

Showing rows 0 - 3 (4 total, Query took 0.0010 seconds.)

```
SELECT * FROM `students`;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	student_id	name	roll_number	department
☐ Edit Copy Delete	1	Hamza	CY101	Cyber Security
☐ Edit Copy Delete	2	Sara	CS102	Computer Science
☐ Edit Copy Delete	3	Bilal	EE103	Electrical Engineering
☐ Edit Copy Delete	4	Fatima	CY104	Cyber Security

☐ Check all With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Output: Four records inserted (verified with `SELECT * FROM students;`)

Step 4: Create the fees Table with Foreign Key

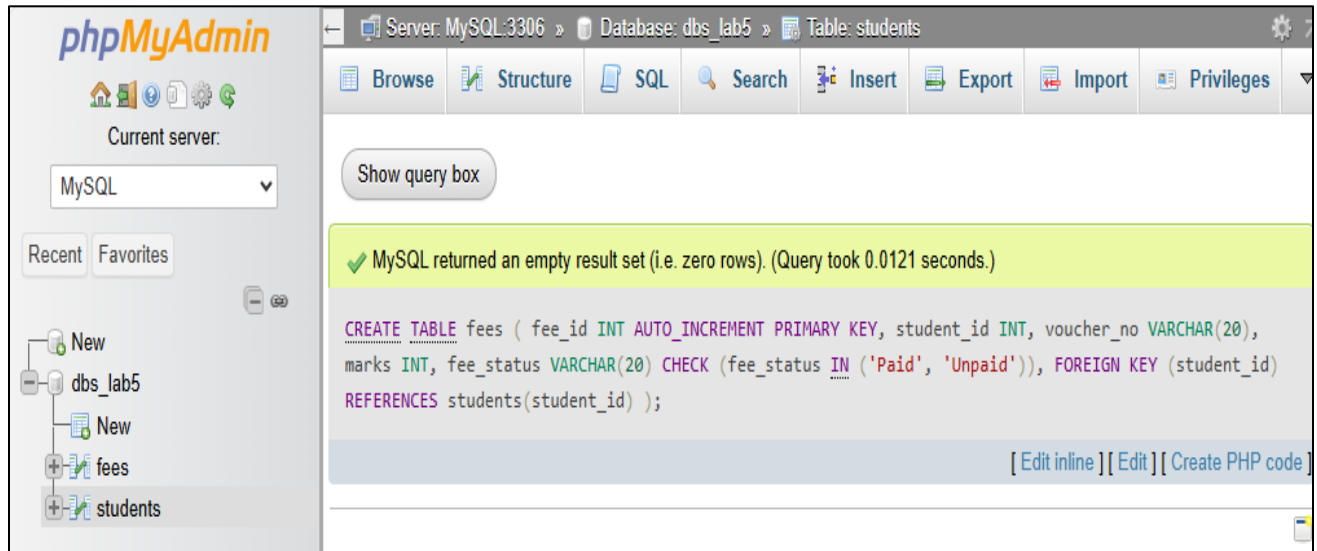
The fees table was created and linked to the students table through a foreign key on student_id.

Query: CREATE TABLE fees (fee_id INT AUTO_INCREMENT PRIMARY KEY, student_id INT, voucher_no VARCHAR(20), marks INT, fee_status VARCHAR(20) CHECK (fee_status IN ('Paid', 'Unpaid')), FOREIGN KEY (student_id) REFERENCES students(student_id));

The screenshot shows the phpMyAdmin web interface. On the left sidebar, the database 'dbs_lab5' is selected, and the 'students' table is highlighted. The main panel displays the 'Structure' tab for the 'students' table, showing columns: student_id, name, roll_number, and department. The 'SQL' tab is active, showing a SQL query to create a new table 'fees'. The query is as follows:

```
1 CREATE TABLE fees (  
2   fee_id INT AUTO_INCREMENT PRIMARY KEY,  
3   student_id INT,  
4   voucher_no VARCHAR(20),  
5   marks INT,  
6   fee_status VARCHAR(20) CHECK (fee_status IN ('Paid',  
7     'Unpaid'))),  
8   FOREIGN KEY (student_id) REFERENCES  
   students(student_id)  
9 );
```

Below the query editor, there are buttons for 'SELECT *', 'SELECT', 'INSERT', 'UPDATE', 'DELETE', 'Clear', 'Format', and '<<'. There is also a 'Get auto-saved query' button and a 'Bind parameters' checkbox.



Output: fees table created with foreign key

Step 5: Insert Records into fees

Four fee records were inserted, each referring to a student_id of the students table.

Query: INSERT INTO fees (student_id, voucher_no, marks, fee_status) VALUES (1, 'VCH-2001', 82, 'Paid'), (2, 'VCH-2002', 74, 'Unpaid'), (3, 'VCH-2003', 91, 'Paid'), (4, 'VCH-2004', 65, 'Unpaid');

The screenshot shows the phpMyAdmin web interface. The top navigation bar indicates the server is MySQL:3306, the database is db5_lab5, and the selected table is fees. The main toolbar includes options like Browse, Structure, SQL, Search, Insert, Export, Import, and More. The left sidebar shows the database structure with 'db5_lab5' selected, containing tables 'fees' and 'students'. The central area is titled 'Run SQL query/queries on table db5_lab5.fees:' and contains the following SQL query:

```
1 INSERT INTO fees (student_id, voucher_no, marks,
2 fee_status) VALUES
3 (1, 'VCH-2001', 82, 'Paid'),
4 (2, 'VCH-2002', 74, 'Unpaid'),
5 (3, 'VCH-2003', 91, 'Paid'),
6 (4, 'VCH-2004', 65, 'Unpaid');
```

Below the query editor, there are buttons for 'SELECT *', 'SELECT', 'INSERT', 'UPDATE', 'DELETE', 'Clear', 'Format', and '<<'. A 'Get auto-saved query' button is also present. At the bottom, there is a checkbox for 'Bind parameters'.

phpMyAdmin

Current server: MySQL

Recent Favorites

New

db5_lab5

New

fees

students

Server: MySQL:3306 » Database: db5_lab5 » Table: fees

Browse Structure SQL Search Insert Export Import Privileges

Show query box

✓ 4 rows inserted.
Inserted row id: 4 (Query took 0.0009 seconds.)

```
INSERT INTO fees (student_id, voucher_no, marks, fee_status) VALUES (1, 'VCH-2001', 82, 'Paid'), (2, 'VCH-2002', 74, 'Unpaid'), (3, 'VCH-2003', 91, 'Paid'), (4, 'VCH-2004', 65, 'Unpaid');
```

[Edit inline] [Edit] [Create PHP code]

phpMyAdmin

Current server: MySQL

Recent Favorites

New

db5_lab5

New

fees

students

Server: MySQL:3306 » Database: db5_lab5 » Table: fees

Browse Structure SQL Search Insert Export Import More

✓ Showing rows 0 - 3 (4 total, Query took 0.0003 seconds.)

```
SELECT * FROM `fees`
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	fee_id	student_id	voucher_no	marks	fee_status
☐ Edit Copy Delete	1	1	VCH-2001	82	Paid
☐ Edit Copy Delete	2	2	VCH-2002	74	Unpaid
☐ Edit Copy Delete	3	3	VCH-2003	91	Paid
☐ Edit Copy Delete	4	4	VCH-2004	65	Unpaid

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Print Copy to clipboard Export Display chart Create view

Output: Four records inserted (verified with `SELECT * FROM fees;`)

Step 6: JOIN Query

A JOIN query was used to display the name, roll number, department, voucher number, marks and fee status of every student in one result.

Query: SELECT students.name, students.roll_number, students.department, fees.voucher_no, fees.marks, fees.fee_status FROM students JOIN fees ON students.student_id = fees.student_id;

The screenshot displays the phpMyAdmin web interface. The top navigation bar shows the server as 'MySQL:3306', the database as 'dbs_lab5', and the selected table as 'fees'. The left sidebar contains a tree view with 'New', 'dbs_lab5', 'New', 'fees', and 'students'. The main panel is titled 'Run SQL query/queries on table dbs_lab5.fees:'. It contains a text area with the following SQL query:

```
1 SELECT students.name, students.roll_number,  
2     students.department,  
3     fees.voucher_no, fees.marks, fees.fee_status  
4 FROM students  
5 JOIN fees ON students.student_id = fees.student_id;
```

To the right of the query editor is a vertical list of column names: fee_id, student_id, voucher_no, marks, and fee_status. Below the query editor, there are buttons for 'SELECT *', 'SELECT', 'INSERT', 'UPDATE', 'DELETE', 'Clear', 'Format', and '<<'. There is also a 'Get auto-saved query' button and a 'Bind parameters' checkbox.

The screenshot shows the phpMyAdmin interface. The left sidebar displays the database structure with 'db5_lab5' selected, containing tables 'fees' and 'students'. The main panel shows the 'Table: fees' view. A message at the top states: 'Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.' Below this, a green bar indicates 'Showing rows 0 - 3 (4 total, Query took 0.0072 seconds.)'. The SQL query is: `SELECT students.name, students.roll_number, students.department, fees.voucher_no, fees.marks, fees.fee_status FROM students JOIN fees ON students.student_id = fees.student_id;`. Below the query, there are links for 'Profiling', 'Edit inline', 'Edit', 'Explain SQL', 'Create PHP code', and 'Refresh'. The table view shows 4 rows with columns: name, roll_number, department, voucher_no, marks, and fee_status. The data is as follows:

name	roll_number	department	voucher_no	marks	fee_status
Hamza	CY101	Cyber Security	VCH-2001	82	Paid
Sara	CS102	Computer Science	VCH-2002	74	Unpaid
Bilal	EE103	Electrical Engineering	VCH-2003	91	Paid
Fatima	CY104	Cyber Security	VCH-2004	65	Unpaid

Below the table, there are controls for 'Show all', 'Number of rows' (set to 25), 'Filter rows' (Search this table), and 'Sort by key' (None). At the bottom, there is a 'Query results operations' button.

Output: Combined data of students and fees

Step 7: JOIN with WHERE (Paid Students)

The joined result was filtered to show only the students whose fee status is Paid.

Query: SELECT students.name, students.roll_number, students.department, fees.voucher_no, fees.marks, fees.fee_status FROM students JOIN fees ON students.student_id = fees.student_id WHERE fees.fee_status = 'Paid';

The screenshot shows the phpMyAdmin web interface. The top navigation bar indicates the server is MySQL:3306, the database is db5_lab5, and the selected table is fees. The left sidebar shows the database structure with 'db5_lab5' containing 'fees' and 'students' tables. The main area displays the 'SQL' tab with a query editor. The query entered is:
1 SELECT students.name, students.roll_number,
students.department,
2 fees.voucher_no, fees.marks, fees.fee_status
3 FROM students
4 JOIN fees ON students.student_id = fees.student_id
5 WHERE fees.fee_status = 'Paid';
6
To the right of the query editor is a table structure preview for the 'fees' table, showing columns: fee_id, student_id, voucher_no, marks, and fee_status. At the bottom, there are buttons for 'SELECT *', 'SELECT', 'INSERT', 'UPDATE', 'DELETE', 'Clear', 'Format', and '<<'. There is also a 'Get auto-saved query' button and a 'Bind parameters' checkbox.

The screenshot shows the phpMyAdmin interface. On the left, the database structure is visible with 'db Lab5' containing 'fees' and 'students' tables. The main panel shows the 'fees' table selected. A warning message at the top states: 'Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.' Below this, a green status bar indicates 'Showing rows 0 - 1 (2 total, Query took 0.0013 seconds.)'. The SQL query is displayed as follows:

```
SELECT students.name, students.roll_number, students.department, fees.voucher_no, fees.marks, fees.fee_status FROM students JOIN fees ON students.student_id = fees.student_id WHERE fees.fee_status = 'Paid';
```

Below the query, there are options for 'Profiling', 'Edit inline', 'Edit', 'Explain SQL', 'Create PHP code', and 'Refresh'. The query results are shown in a table with the following data:

name	roll_number	department	voucher_no	marks	fee_status
Hamza	CY101	Cyber Security	VCH-2001	82	Paid
Bilal	EE103	Electrical Engineering	VCH-2003	91	Paid

At the bottom, there are options for 'Show all', 'Number of rows' (set to 25), 'Filter rows' (Search this table), and 'Sort by key' (None). The 'Query results operations' section includes buttons for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'.

Output: Students with Paid fee status

Step 8: JOIN with Multiple Conditions

The result was filtered further to show only Paid students of the Cyber Security department.

Query: SELECT students.name, students.roll_number, students.department, fees.voucher_no, fees.marks, fees.fee_status FROM students JOIN fees ON students.student_id = fees.student_id WHERE fees.fee_status = 'Paid' AND students.department = 'Cyber Security';

The screenshot shows the phpMyAdmin web interface. The top navigation bar indicates the server is MySQL:3306, the database is dbs_lab5, and the table is fees. The left sidebar shows the database structure with 'dbs_lab5' selected, containing tables 'fees' and 'students'. The main area displays the SQL query editor with the following query:

```
1 SELECT students.name, students.roll_number,  
2     students.department,  
3     fees.voucher_no, fees.marks, fees.fee_status  
4 FROM students  
5 JOIN fees ON students.student_id = fees.student_id  
6 WHERE fees.fee_status = 'Paid'  
7     AND students.department = 'Cyber Security';
```

Below the query editor, there are buttons for 'SELECT *', 'SELECT', 'INSERT', 'UPDATE', 'DELETE', 'Clear', 'Format', and '<<'. There is also a 'Get auto-saved query' button and a 'Bind parameters' checkbox.

The screenshot shows the phpMyAdmin interface. The left sidebar displays the database structure with 'dbs_lab5' selected, containing tables 'fees' and 'students'. The main panel shows the 'fees' table selected. A message at the top states: 'Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.' Below this, a green bar indicates 'Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)'. The SQL query is: `SELECT students.name, students.roll_number, students.department, fees.voucher_no, fees.marks, fees.fee_status FROM students JOIN fees ON students.student_id = fees.student_id WHERE fees.fee_status = 'Paid' AND students.department = 'Cyber Security';`. Below the query, there are options for 'Show all', 'Number of rows: 25', and 'Filter rows: Search this table'. An 'Extra options' button is also present. The query results are displayed in a table with the following data:

name	roll_number	department	voucher_no	marks	fee_status
Hamza	CY101	Cyber Security	VCH-2001	82	Paid

Below the table, there are again options for 'Show all', 'Number of rows: 25', and 'Filter rows: Search this table'. At the bottom, a 'Query results operations' section includes buttons for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'.

Output: Paid students of the Cyber Security department

Results:

The students and fees tables were created successfully, and fees.student_id was linked to students.student_id as a foreign key. Four student records and four fee records were inserted without any error.

The JOIN query returned all four students together with their department, voucher number, marks and fee status in a single table. Filtering with fee_status = 'Paid' reduced the result to two students, Hamza and Bilal. Adding the condition department = 'Cyber Security' reduced it further to one student, Hamza, which confirms that the JOIN and the WHERE conditions worked correctly together.

Conclusion:

This lab showed how a foreign key creates a relationship between two tables and how a JOIN query brings the related data together. Keeping student records and fee records in separate tables avoids repeating student information for every fee entry. The queries also showed that WHERE conditions can be added on top of a JOIN to answer more specific questions about the combined data.